Automatic Differentiation

ES 335 Machine Learning, Semester I 2025–26 Sources: lectures "Automatic differentiation" and "Autograd from scratch", cheat sheet ML18, and the autodiff and autograd notebooks

What this file is really about:

$$\text{a loss is a chain of simple steps} \rightarrow \text{record them} \rightarrow \text{walk back} \rightarrow \text{gradients add} \rightarrow \text{one sweep for}$$

How to use this file. Part A works through a logistic loss node by node. Part B builds the engine in Python. Part C is practice with answers.

Step-up path

You need. The chain rule and the multi-layer perceptron.

1. Record a loss as a graph of simple operations.
2. Multiply upstream by local, backwards.
3. Add gradients where paths meet.
4. Compare forward and reverse mode.
5. Build a small engine.

You can stop when you can run backpropagation on paper and explain why it needs only one sweep.

Contents

Part A Maths From the Ground Up

A1 Why not nudge each parameter

A numerical derivative nudges one input:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}.$$

This costs two evaluations per parameter, so a million parameters need two million forward passes. It is also fragile: at $f(x) = 3x^2 + 2x + 1$ and $x = 2$ the true slope is 14, but a tiny h subtracts two nearly equal numbers and rounding error dominates. At $h = 10^{-20}$, $x + h$ equals x exactly in floating point, and the estimate is 0.

Keep numerical derivatives for checking gradients.

Autodiff is not numerical differentiation. It is not symbolic differentiation either. It applies the chain rule to the actual numbers as the program runs, and it is exact up to floating-point rounding.

A2 A computation graph

Any loss is built from a few primitive operations, each with a known local derivative:

Operation	Local gradient
$a + b$	$1,\ 1$
$a \cdot b$	$b,\ a$
e^a	e^a
$1/a$	$-1/a^2$
$\log a$	$1/a$
$\sigma(a)$	$\sigma(1-\sigma)$

Record the operations in a graph on the way forward, storing each value.

A3 Downstream equals upstream times local

For a node $z = f(x, y)$ the chain rule says

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial x}.$$

$\partial L/\partial z$ is the **upstream** gradient, and $\partial z/\partial x$ is the **local** one. Start at the loss with $\partial L/\partial L = 1$ and walk back through the nodes in reverse order. A library only needs each primitive's local gradient.

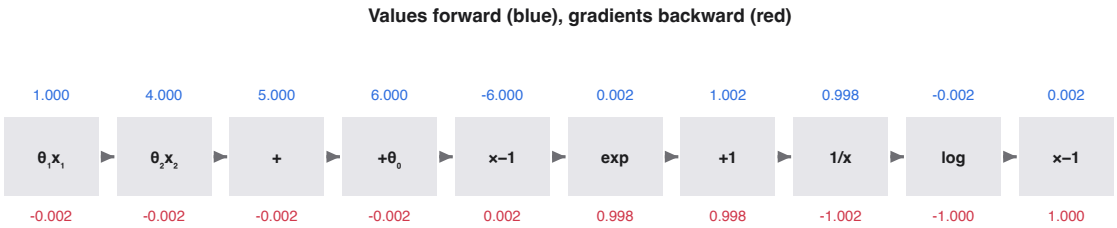
A4 The logistic loss, node by node

Take $y = 1$, $x = (1, 2)$ and $\theta = (1, 1, 2)$. Then $z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 = 6$ and $L = \log(1 + e^{-z}) = 0.00248$.

Node	Value	$\partial L/\partial \text{node}$
$f_1 = \theta_1 x_1$	1	-0.00247
$f_2 = \theta_2 x_2$	4	-0.00247
$f_3 = f_1 + f_2$	5	-0.00247
$f_4 = f_3 + \theta_0$	6	-0.00247
$f_5 = -f_4$	-6	0.00247
$f_6 = e^{f_5}$	0.00248	0.9975
$f_7 = f_6 + 1$	1.00248	0.9975
$f_8 = 1/f_7$	0.99753	-1.0025
$f_9 = \log f_8$	-0.00248	-1
$L = -f_9$	0.00248	1

So $\partial L/\partial \theta_0 = -0.00247$, and because θ_2 is multiplied by $x_2 = 2$, $\partial L/\partial \theta_2 = -0.00495$.

Check. This equals $(\sigma(z) - y)x_j$, the logistic-regression gradient.



Each node shows its value from the forward pass and its gradient from the backward pass.

A5 A reused value adds its gradients

If one value is used twice, it receives a gradient along each path, and these add. For $z = (x + y)(x - y)$ at $x = 3$, $y = 2$:

$$\frac{\partial z}{\partial x} = 1 \cdot 1 + 5 \cdot 1 = 6, \qquad \frac{\partial z}{\partial y} = 1 \cdot 1 + 5 \cdot (-1) = -4.$$

Parameters shared across training examples accumulate gradients in the same way.

> **Careful.** Gradients accumulate until you clear them. Without `zero_grad()`, each step adds to the previous ones and training goes wrong.

A6 Forward mode against reverse mode

Forward mode carries a value and its derivative together, using dual numbers $a + \dot{a}\varepsilon$ with $\varepsilon^2 = 0$:

$$(a + \dot{a}\varepsilon)(b + \dot{b}\varepsilon) = ab + (\dot{a}b + a\dot{b})\varepsilon.$$

One sweep gives the derivative for one input. **Reverse mode** gives the derivative of one output with respect to all inputs in a single sweep.

For $f : \mathbb{R}^n \to \mathbb{R}^m$, a forward sweep gives one Jacobian column, and a reverse sweep one row. A network has one scalar loss and millions of parameters, so it uses reverse mode, costing roughly four forward passes regardless of n. The price is storing every intermediate value.

A7 Backpropagation is reverse mode

Applied entry by entry to layers, reverse mode gives the matrix formulas of the previous note: each gradient has the same shape as its parameter.

A8 Forward mode with dual numbers

Write $x + \epsilon\dot{x}$, with the rule $\epsilon^2 = 0$. Then

$$f(x + \epsilon\dot{x}) = f(x) + \epsilon f'(x)\dot{x},$$

so evaluating with these pairs computes the derivative automatically, for one input direction per pass.

A9 The cost of each mode

For $f : \mathbb{R}^n \to \mathbb{R}^m$, forward mode needs n passes and reverse needs m. Networks have n in the millions and $m = 1$, which is why reverse mode wins. Reverse mode's price is storing the intermediate values.

A10 Second derivatives without a Hessian

A full Hessian has n^2 entries, which is impossible for a big network. A **Hessian-vector product** needs only two gradient computations:

$$Hv = \nabla_w\big(\nabla_w L \cdot v\big).$$

These products power curvature estimates and second-order methods.

A11 Numerical gradient checks

To test an implementation, compare with central differences:

$$\frac{\partial L}{\partial w_i} \approx \frac{L(w + he_i) - L(w - he_i)}{2h}.$$

The error is $O(h^2)$. Use $h \approx 10^{-5}$ and double precision.

Part B In Code

B1 A value that remembers its past

```
class Value:
    def __init__(self, data, prev=()):
        self.data, self.prev, self.grad = data, prev, 0.0
        self._back = lambda: None

    def __mul__(self, o):
        out = Value(self.data * o.data, (self, o))
        def _back():
            self.grad += o.data * out.grad
            o.grad += self.data * out.grad
        out._back = _back
        return out
```

B2 Walking back

```
def backward(self):
    order = topological_order(self)   # inputs before outputs
    self.grad = 1.0
    for v in reversed(order):
        v._back()
```

B3 The PyTorch version

```
opt.zero_grad()                  # clear old gradients
loss = loss_fn(model(X), y)
loss.backward()                  # fill .grad
opt.step()                       # theta -= lr * grad
```

The notebook **Autograd from scratch** in this section builds the whole engine, and **Automatic differentiation** prints the same numbers as the table above.

B4 Forward mode in a dozen lines

```python
import math

class Dual:
    def __init__(self, v, d=0.0):
        self.v, self.d = v, d
    def __add__(a, b):
        b = b if isinstance(b, Dual) else Dual(b)
        return Dual(a.v + b.v, a.d + b.d)
    __radd__ = __add__
    def __mul__(a, b):
        b = b if isinstance(b, Dual) else Dual(b)
        return Dual(a.v * b.v, a.d * b.v + a.v * b.d)
    __rmul__ = __mul__

def sin(x):
    return Dual(math.sin(x.v), math.cos(x.v) * x.d)

x = Dual(2.0, 1.0)
y = x * x * 3 + sin(x)
print(y.v, y.d)
assert abs(y.d - (6 * 2 + math.cos(2))) < 1e-12
```

B5 A Hessian-vector product

```
import torch

w = torch.tensor([1.0, 2.0], requires_grad=True)
L = w[0] ** 2 * w[1] + w[1] ** 3
g = torch.autograd.grad(L, w, create_graph=True)[0]
v = torch.tensor([1.0, 0.0])
hv = torch.autograd.grad(g @ v, w)[0]
print(hv)                                   # first column of [[2w1, 2w0], [2w0, 6w1]]
assert hv.tolist() == [4.0, 2.0]
```

B6 A gradient check

```
import torch
from torch.autograd import gradcheck
from torch.nn import functional as F

f = lambda w: F.softplus(w).sum() + (w ** 2).mean()
w = torch.randn(5, dtype=torch.double, requires_grad=True)
assert gradcheck(f, (w,))
print('analytic gradient matches numeric')
```

Part C Practice

P1. $f = (x + y)z$ at $x = 2$, $y = 3$, $z = 4$. Find $\partial f/\partial x$ and $\partial f/\partial z$.

$\partial f/\partial x = z = 4$, and $\partial f/\partial z = x + y = 5$.

P2. $L = \theta x_1 + \theta x_2$ with $x = (1, 2)$. Find $\partial L/\partial \theta$ and name the rule.

$1 + 2 = 3$. Gradients along different paths add.

P3. Three calls to `backward()` on that loss without zeroing. What does `.grad` hold?

9, because gradients accumulate: 3, 6, then 9.

P4. A network has $61{,}706$ weights. How many backward passes give every gradient?

One.

P5. Multiply dual numbers $(2 + 3\varepsilon)(4 + \varepsilon)$.

$8 + (2 + 12)\varepsilon = 8 + 14\varepsilon$.

P6. Upstream gradient 5 arrives at $z = x^2$ with $x = 3$. What does x receive?

$5 \times 2x = 30$.

P7. A function maps 3 inputs to 1000 outputs. Which mode would you use for its full Jacobian?

Forward mode, with 3 passes rather than 1000.

P8. Evaluate x^2 with the dual number $3 + \epsilon$.

$9 + 6\epsilon$, so the derivative is 6.